Read Replicas & Replication Lag

fixing stale reads after writes

Watch the 4-min video

In one line: read replicas scale reads, but with async replication they lag, so a user can miss their own write. Route a user's fresh data to the primary (always, or for a short window, or until a replica has replayed the write's log position), and pin users to one replica so time never goes backwards.

1 The problem: “I saved it, why is it the old name?”

A user changes their display name and clicks save. The page reloads and shows the old name. They save again. Still the old name. Nothing is broken: the write went to the primary, but the read went to a replica that was half a second behind.

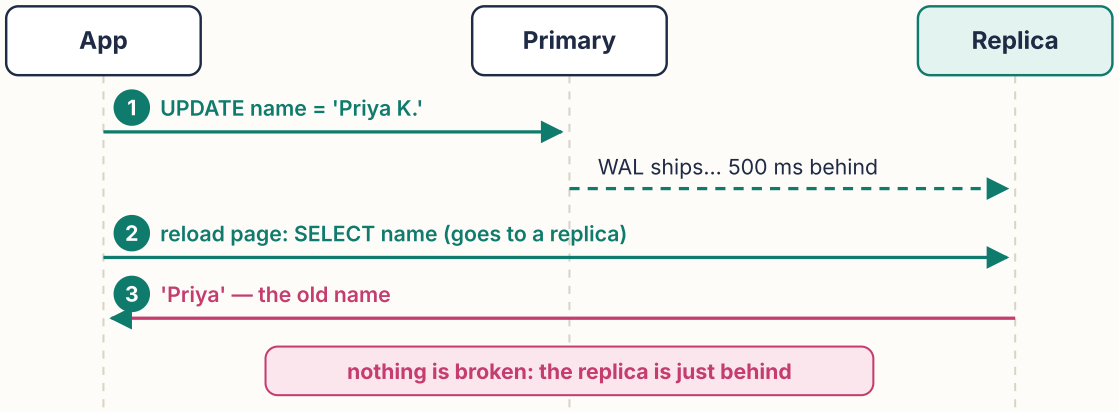


Figure 1. The write succeeded on the primary. The very next read hit a replica that had not replayed it yet.

2 Read replicas: the easiest way to scale reads

Writes go to the primary. Reads are spread across replicas: by the application, by a proxy (ProxySQL, Pgpool), or through a **reader endpoint**. Amazon RDS and Aurora give you a reader endpoint that spreads connections across the replicas for you.

Most apps read far more than they write, so this goes a long way. Two limits:

- **No help with write throughput.** Every replica must still apply every write.
- **Every replica lags a little** with async replication: usually milliseconds, sometimes seconds under load.

WORKED EXAMPLE: WHAT REPLICAS BUY YOU

An app does **10,000 queries/s, 90% reads**. Put 9,000 reads/s across **3 replicas**: 3,000 each. The primary now handles only the **1,000 writes/s**, plus any reads you deliberately send to it. Add a fourth replica and reads per node drop to 2,250. But double the writes to 2,000/s and every node, replicas included, must apply 2,000 writes/s: replicas cannot fix that.

3 Three classic lag anomalies

- **Read-your-writes** (read-after-write): you do not see your own update, like the display name.
- **Monotonic reads:** two requests hit different replicas, and time seems to go backwards. A comment appears, then disappears on refresh.
- **Consistent prefix:** you see an answer before the question it replies to.

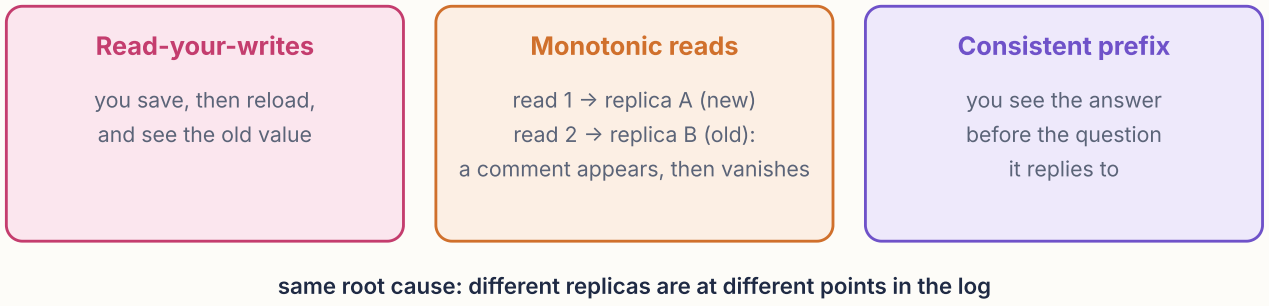


Figure 2. Three symptoms, one cause: each replica has replayed a different amount of the log.

4 Fix one: read from the primary when it matters

- **Always** read the user's own profile and settings from the primary. Other people's profiles can come from replicas.
- **Or use a short window:** after any write, set a flag in the session (for example for 5 seconds, comfortably above normal lag), and send that user's reads to the primary until it expires.

Simple, but it adds load to the primary. It works well when users mostly read their own data, like settings pages and dashboards.

5 Fix two: track positions in the log

After a write, record the primary's current log position, the **LSN** (log sequence number), in the user's session. On the next read, use a replica that has replayed **at least that far**. If none has, wait briefly, or fall back to the primary.

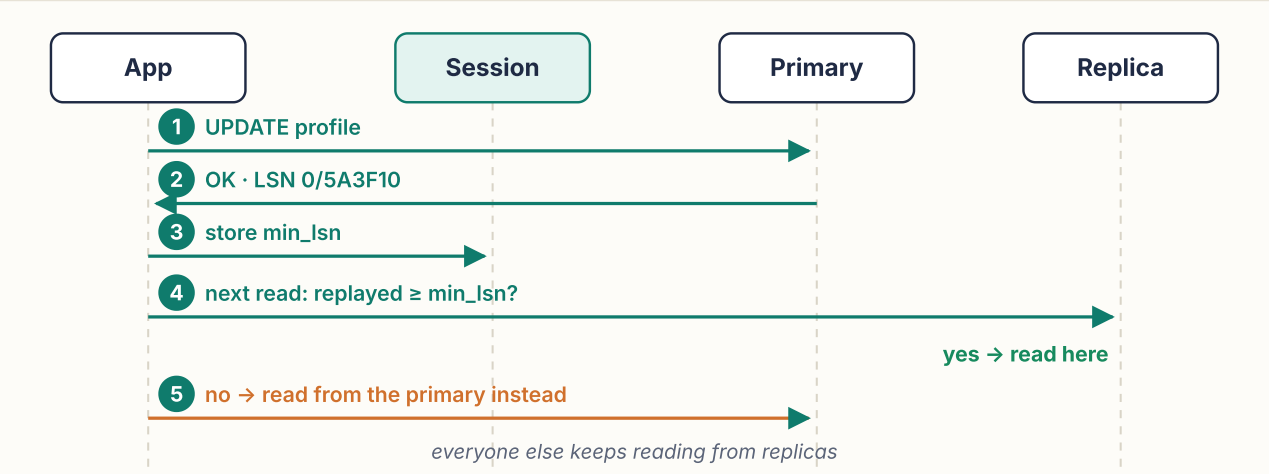


Figure 3. The profile flow. The user always sees the new name, and the primary only serves the reads that truly need it.

- **Postgres:** `pg_current_wal_lsn()` on the primary after the write; `pg_last_wal_replay_lsn()` on a replica to see how far it has replayed.
- **MySQL:** the same idea with **GTIDs** (global transaction IDs): `WAIT_FOR_EXECUTED_GTID_SET` waits until a replica has applied them.

6 Monotonic reads and consistent prefix

For **monotonic reads**, pin each user to one replica, for example by hashing the user ID. They may see slightly old data, but never go backwards. If that replica fails, rehash to another.

Consistent prefix is mostly a problem in partitioned (sharded) systems, where a question and its answer can live on different partitions with different lag. It is solved by writing causally related data, like a thread and its replies, to the same partition.

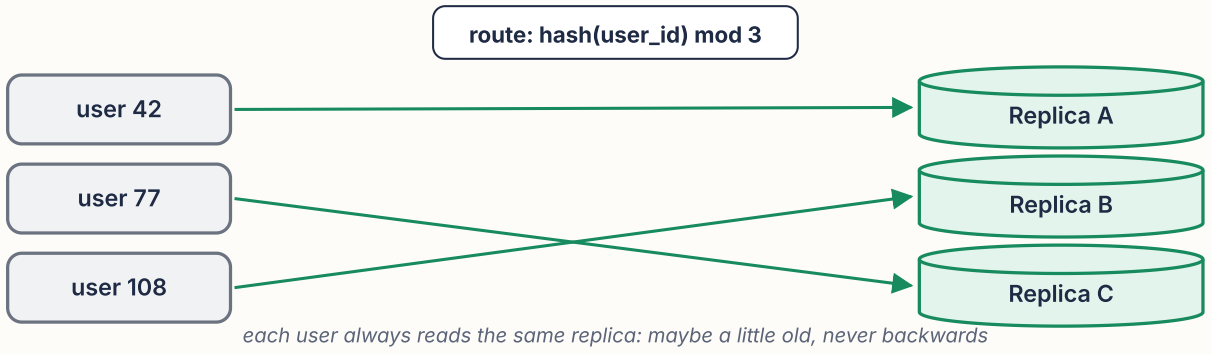


Figure 4. Sticky replicas: a deterministic hash sends each user to the same replica every time.

Most setups combine both: **primary reads for a user's own fresh data**, and **sticky replicas for everything else**.

COMMON MISTAKES

- Sending every read through a round-robin reader endpoint, including the read right after a write.
- Sending *all* reads to the primary "to be safe", which throws away the replicas.
- A read-from-primary window shorter than real lag under load (measure p99 lag, not the average).
- Expecting replicas to fix a write bottleneck.

7 Key terms

Read replica: a follower that serves reads

Read-your-writes: you always see your own updates

LSN: Postgres log sequence number

Reader endpoint: one address spread over replicas

Monotonic reads: time never goes backwards for you

GTID: MySQL global transaction ID

Replication lag: how far a replica is behind

Consistent prefix: you see events in causal order

Sticky replica: a user pinned to one replica

8 Interview questions

Q1. How do you avoid stale reads after a write? BASIC

Route reads of a user's own recently changed data to the primary, always or for a short window after a write, or track the write's log position and read only from replicas that have replayed it. Pin users to one replica for monotonic reads.

Q2. Do read replicas increase write throughput? BASIC

No. Every replica must apply every write, so writes are still limited by the primary (and each replica must keep up). Replicas scale reads only; for writes you need sharding or a faster primary.

Q3. What is the monotonic reads anomaly? BASIC
Two reads go to different replicas with different lag, so the second shows older data than the first: a comment appears, then disappears. Fix it by pinning each user to one replica.

Q4. Users say a just-placed order is missing from "My orders" for about a second. Fix? SCENARIO
Classic read-your-writes. After the order write, store the primary's LSN (or a 5-second flag) in the session; serve "My orders" from a replica only if it has replayed that LSN, otherwise from the primary. Other users' reads stay on replicas.

Q5. Your app does 20,000 reads/s and 2,000 writes/s. One replica handles about 6,000 reads/s. How many replicas? SCENARIO
20,000 / 6,000 ≈ 3.3, so 4 replicas, plus one spare for failure and for lag spikes: 5. Check that each can also apply 2,000 writes/s without lagging.

Q6. During a nightly batch, replica lag reaches 30 s. What happens to your 5-second primary-read window? SCENARIO
It is too short: users whose window expired read from replicas still 30 s behind and see stale data. LSN tracking handles this correctly. Also throttle the batch, and stop routing to replicas whose lag exceeds a threshold.

Q7. Why not just send all reads to the primary? TRICKY
Then replicas only help with failover, and the primary carries the full read load, which was the problem. Send only the reads that need fresh, own data to the primary; often well under 10% of reads.

Q8. A user is pinned to replica B, which fails. They rehash to replica A. Can they see time go backwards? TRICKY
Yes, once, if A is further behind than B was. To avoid it, store the last LSN the user saw and only switch to a replica at or past it (or read from the primary briefly).

Next: SD-E-021 · Multi-Leader & Leaderless Replication backendroadmap.com